

Task7：自制数据集并训练一次 YOLOv8 模型

一、任务要求

使用提供的场地彩球照片**自行制作数据集并完成标注**，然后训练一次 YOLOv8 目标检测模型，具体要求如下：

1. 数据集为提供的 56 张比赛场地实拍照片（1.jpg ~ 56.jpg，分辨率均为 1280×720）；
2. 按 data.yaml 中给定的类别顺序完成标注，共 3 个类别：

类别 ID	名称	目标
0	blue	蓝色球
1	red	红色球（印有 BJTU）
2	invalid	紫色 / 品红色球（印有 UP，视为无效球）

3. 训练一次 YOLOv8 模型，使其能够检测并区分上述三类球；
4. 需要提交：训练得到的权重文件 .pt、训练结果图 results.png、电脑运行截图以及解释文档。

实验环境：Windows 11 + Python 3.11.16（虚拟环境）+ PyTorch 2.11.0+cu128（CUDA 12.8）+ Ultralytics 8.4.155 + OpenCV 5.0.0，显卡 NVIDIA GeForce RTX 5060 Laptop GPU（8 GB，Blackwell 架构）。

二、实验原理

2.1 YOLOv8 检测原理

YOLO（You Only Look Once）是典型的**单阶段（one-stage）目标检测算法**：它把输入图片划分成网格，由一个**卷积神经网络一次性**同时预测出每个目标的**类别和边界框（bounding box）**，不需要两阶段方法中的“先找候选框再分类”过程，因此速度快、适合实时检测。

YOLOv8 的整体结构分为三部分：

- **Backbone（骨干网络）**：采用 CSP 思想的 C2f 模块，提取图像的多尺度特征；
- **Neck（颈部）**：采用 PAN + FPN 结构，融合浅层（小目标、位置）与深层（语义、类别）特征；
- **Head（检测头）**：Anchor-Free 的解耦头，分别输出分类分数与回归框，并在 80×80、40×40、20×20 三个尺度上检测小、中、大目标。

本实验使用参数量最小的 **YOLOv8n（nano）**，在 COCO 预训练权重 yolov8n.pt 的基础上进行迁移学习，仅 3.0 M 参数、8.1 GFLOPs，在小数据集上也能快速收敛。

2.2 YOLO 标注格式

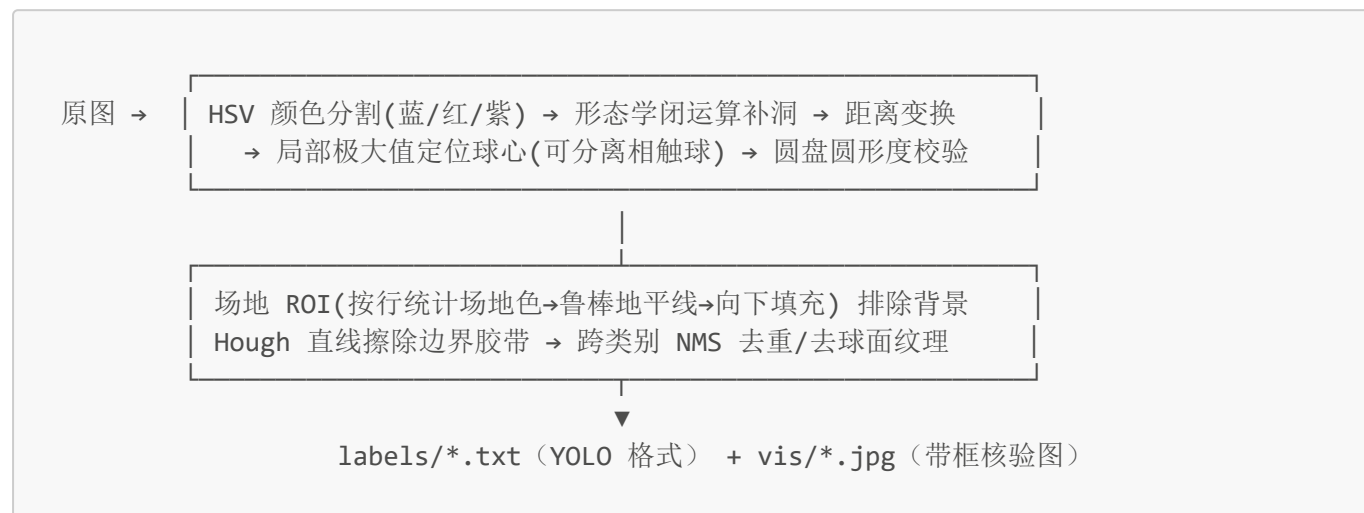
YOLO 使用每行一个目标的 .txt 标签文件，与图片同名，字段全部**归一化到 0~1**：

```
<class_id> <center_x> <center_y> <width> <height>
```

其中 `center_x / center_y` 为框中心相对图片宽高的比例，`width / height` 为框宽高相对图片宽高的比例。一张图对应一个标签文件，没有目标的图片对应空文件。

2.3 自动标注思路

56 张图片若全部手工框选工作量大且不一致。观察到三类球颜色高度饱和、与绿色 / 青色场地对比强烈，因此采用**传统视觉自动标注 + 逐张可视化核验**的方案生成标签，流程如下：



关键点：

- **HSV 分割**：在 HSV 空间用色相（H）区间区分蓝、红、紫，比 RGB 更能抵抗光照明暗变化；
- **距离变换 + 局部极大值**：对每个颜色连通域做距离变换，局部极大值即球心，即使两颗球相触也能分开；
- **圆形度 / 填充率校验**：真球在球心附近近似实心圆盘，胶带、横幅、桌面杂物呈带状或空心，可据此剔除；
- **场地 ROI**：只保留比赛场地（含黄色地板）内的目标，自动排除红色横幅、实验台、人物、设备等背景。

三、运行环境搭建

3.1 创建 Python 3.11 虚拟环境

RTX 5060 属于 Blackwell 架构，需要 CUDA 12.8（cu128）版本的 PyTorch。这里用 `uv` 安装 Python 3.11 并在任务目录下创建虚拟环境：

```
# 安装 uv（已安装可跳过）
python -m pip install uv -i https://pypi.tuna.tsinghua.edu.cn/simple

# 安装 Python 3.11 并创建虚拟环境
uv python install 3.11
uv venv --python 3.11 .venv

# 激活虚拟环境（Windows PowerShell）
.\.venv\Scripts\Activate.ps1
```

3.2 安装 GPU 版 PyTorch (cu128)

从 PyTorch 官方 cu128 索引安装与 Blackwell 显卡匹配的 torch / torchvision:

```
uv pip install --python .\.venv\Scripts\python.exe ^
  "https://download.pytorch.org/whl/cu128/torch-2.11.0%2Bcu128-cp311-cp311-
  win_amd64.whl" ^
  "https://download.pytorch.org/whl/cu128/torchvision-0.26.0%2Bcu128-cp311-cp311-
  win_amd64.whl" ^
  --index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

注意：默认 PyPI / 清华源装到的是 CPU 版 (2.11.0+cpu, torch.cuda.is_available() 为 False)，必须显式安装带 +cu128 的轮子才能调用 5060 显卡。

3.3 安装 Ultralytics 与 OpenCV

```
uv pip install --python .\.venv\Scripts\python.exe ^
  ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
```

安装完成后验证 GPU 可用 (训练终端顶部也会打印同样信息):

```
python -c "import
torch;print(torch.__version__,torch.cuda.is_available(),torch.cuda.get_device_name
(0))"
# 2.11.0+cu128 True NVIDIA GeForce RTX 5060 Laptop GPU
```

四、数据集与自动标注

4.1 原始数据

提供的数据解压后目录如下, data.yaml 给出了类别, 原始图片位于 Task7/images/:

```
Task7/
├─ data.yaml          # 类别定义 (路径为占位符, 需自行重建)
├─ images/            # 1.jpg ~ 56.jpg, 共 56 张, 1280×720
```

原始 data.yaml 内容 (类别顺序即标注 ID 顺序):

```
train: C:\path\to\your\train
val:   C:\path\to\your\valid
test:  \path\to\your\test
```

```
nc: 3
names:
  0: blue
  1: red
  2: invalid
```

56 张原图总览（含近景特写、密集堆叠、手放球、广角远景等多种场景）：



4.2 自动标注脚本 auto_label.py

脚本读取 `Task7/images/` 下全部图片，按 2.3 节流程生成 YOLO 标签到 `labels/`，同时在 `vis/` 输出带框核验图。完整代码如下：

```
# -*- coding: utf-8 -*-
"""
Task7 自动标注脚本
1) HSV 分割蓝/红/紫三类高饱和球（绿色场地  $H < 105$  被排除）；
2) 距离变换 + 局部极大值定位每颗球心（可分离相互接触的球）；
3) 球心圆盘圆形度校验，剔除带状/碎块干扰；
4) 由绿色/青色场地按行统计生成“场地 ROI”（含黄色地板与围挡边），
   只保留位于比赛场地内的球，自动排除横幅、桌面杂物等背景；
5) 输出 YOLO 标签与带框核验图。
类别顺序严格按 data.yaml: 0 blue, 1 red, 2 invalid(紫球)
"""
import os
```

```

import cv2
import numpy as np

ROOT = os.path.dirname(os.path.abspath(__file__))
IMG_DIR = os.path.join(ROOT, "Task7", "images")
LABEL_DIR = os.path.join(ROOT, "labels")
VIS_DIR = os.path.join(ROOT, "vis")
os.makedirs(LABEL_DIR, exist_ok=True)
os.makedirs(VIS_DIR, exist_ok=True)

# OpenCV H:0-179 S:0-255 V:0-255
COLOR_RANGES = {
    0: {"name": "blue", "ranges": [((105, 55, 45), (138, 255, 255))]},
    1: {"name": "red", "ranges": [((0, 55, 45), (10, 255, 255)),
                                   ((168, 55, 45), (179, 255, 255))]},
    2: {"name": "invalid", "ranges": [((139, 45, 30), (167, 255, 255))]},
}

DRAW_COLOR = {0: (255, 128, 0), 1: (0, 0, 255), 2: (200, 0, 200)} # BGR

MIN_RADIUS = 9.0
MAX_RADIUS = 175.0
PEAK_REL = 0.50
PEAK_SEP_REL = 0.62
DISK_FRAC = 0.68
BOX_FILL = 0.45 # 方形框内该颜色像素占比下限
BOX_SCALE = 1.06
FIELD_MARGIN = 52 # 地平线上方扩张像素，覆盖贴围挡的球
DEBUG_MASK = os.environ.get("DEBUG_MASK")

def suppress_lines(mask):
    """擦除场地边界细长直胶带线：在 Canny 边缘上找长直线，
    并沿线采样距离变换，确认各处都很细(胶带)才擦除，避免误删大球。"""
    H, W = mask.shape
    dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
    edges = cv2.Canny(mask, 50, 150)
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=40,
                             minLineLength=120, maxLineGap=25)
    if lines is not None:
        for (x1, y1, x2, y2) in lines.reshape(-1, 4):
            if ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5 < 120:
                continue
            thin = True
            for t in (0.25, 0.5, 0.75):
                px, py = int(x1 + t * (x2 - x1)), int(y1 + t * (y2 - y1))
                if not (0 <= px < W and 0 <= py < H) or dist[py, px] >= 13:
                    thin = False
                    break
            if thin:
                cv2.line(mask, (x1, y1), (x2, y2), 0, 20)
    return mask

def build_color_mask(hsv, cls):

```

```

m = None
for lo, hi in COLOR_RANGES[cls]["ranges"]:
    part = cv2.inRange(hsv, np.array(lo, np.uint8), np.array(hi, np.uint8))
    m = part if m is None else cv2.bitwise_or(m, part)
k3 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
k7 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, k7, iterations=2)
m = cv2.morphologyEx(m, cv2.MORPH_OPEN, k3, iterations=1)
m = suppress_lines(m)          # 擦除贴在胶带边上的细长反光
m = cv2.dilate(m, k3, iterations=1)
return m

```

```
def build_field_roi(hsv):
```

```

    """鲁棒场地 ROI: 冷色灰墙/杂物也可能落入青绿色相, 故不逐列直接取顶,
    而是按行统计场地色占比, 找到“持续高占比”的场地起始带(地平线),
    再在其邻域内逐列细化并向下填充, 把 ROI 严格限制在场地(含黄色地板)内。"""
    H, W = hsv.shape[:2]
    green = cv2.inRange(hsv, np.array((32, 32, 22), np.uint8),
                        np.array((104, 255, 255), np.uint8))
    green = cv2.morphologyEx(
        green, cv2.MORPH_CLOSE,
        cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (45, 45)), iterations=1)
    rowfrac = green.mean(axis=1) / 255.0
    sm = cv2.boxFilter(rowfrac, -1, (1, 31)).ravel()
    hg = None
    for y in range(H - 70):
        if sm[y] > 0.55 and np.median(sm[y:y + 70]) > 0.50:
            hg = y
            break
    if hg is None:          # 近景图场地几乎占满画面
        has = green.any(axis=0)
        if has.sum() < W * 0.3:
            return np.full((H, W), 255, np.uint8)
        hg = int(np.median([np.where(green[:, x] > 0)[0].min()
                           for x in np.where(has)[0]]))
    # 地平线邻域内逐列细化
    tops = np.full(W, hg, dtype=np.float32)
    for x in range(W):
        band = green[max(0, hg - 40):hg + 90, x]
        idx = np.where(band > 0)[0]
        if len(idx):
            tops[x] = idx.min() + max(0, hg - 40)
    tops = cv2.medianBlur(tops.astype(np.uint8).reshape(1, -1), 101) \
        .ravel().astype(np.float32)
    tops = np.clip(tops, hg - 45, hg + 90)
    roi = np.zeros((H, W), np.uint8)
    for x in range(W):
        top = int(max(0, tops[x] - FIELD_MARGIN))
        roi[top:, x] = 255
    roi = cv2.dilate(roi, cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (31, 7)), iterations=1)
    return roi

```

```

def find_peaks(mask):
    H, W = mask.shape
    n, labels, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)
    raw = []
    for k in range(1, n):
        if stats[k, cv2.CC_STAT_AREA] < 120:
            continue
        x0, y0, w, h = (stats[k, cv2.CC_STAT_LEFT], stats[k, cv2.CC_STAT_TOP],
                        stats[k, cv2.CC_STAT_WIDTH], stats[k, cv2.CC_STAT_HEIGHT])
        roi = (labels[y0:y0 + h, x0:x0 + w] == k).astype(np.uint8)
        dist = cv2.distanceTransform(roi, cv2.DIST_L2, 5)
        rmax = float(dist.max())
        if rmax < MIN_RADIUS:
            continue
        sep = max(3, int(round(PEAK_SEP_REL * rmax)))
        dil = cv2.dilate(dist, np.ones((2 * sep + 1, 2 * sep + 1), np.uint8))
        for (py, px) in np.argwhere((dist == dil) & (dist >= PEAK_REL * rmax)):
            r = float(dist[py, px])
            if MIN_RADIUS <= r <= MAX_RADIUS:
                raw.append((float(x0 + px), float(y0 + py), r))
    # 同色峰点去重
    raw.sort(key=lambda t: -t[2])
    out = []
    for (cx, cy, r) in raw:
        if all(((cx - kx) ** 2 + (cy - ky) ** 2) ** 0.5 >= 0.6 * min(r, kr)
              for (kx, ky, kr) in out):
            out.append((cx, cy, r))
    return out

def disk_fraction(mask, cx, cy, r):
    H, W = mask.shape
    rr = max(2.0, 0.72 * r)
    x1, x2 = int(max(0, cx - rr)), int(min(W - 1, cx + rr))
    y1, y2 = int(max(0, cy - rr)), int(min(H - 1, cy + rr))
    yy, xx = np.ogrid[y1:y2 + 1, x1:x2 + 1]
    circ = (xx - cx) ** 2 + (yy - cy) ** 2 <= rr ** 2
    if circ.sum() == 0:
        return 0.0
    return float(np.count_nonzero(np.logical_and(
        mask[y1:y2 + 1, x1:x2 + 1] > 0, circ))) / circ.sum()

def nms(dets):
    """去重：同类近距离合并；异类时若小框中心落在大框内部则丢弃小框。"""
    keep = []
    for d in sorted(dets, key=lambda x: -x[3]):
        cls, cx, cy, r = d
        drop = False
        for (kc, kx, ky, kr) in keep:
            dist = ((cx - kx) ** 2 + (cy - ky) ** 2) ** 0.5
            if cls == kc and (dist < 0.7 * min(r, kr) or dist < 0.78 * kr):
                drop = True # 小框中心落入大球内部(球内重复峰)
        if not drop:
            keep.append(d)

```

```

        break
    if cls != kc:
        # 小框中心位于异类大框范围内且半径不足0.45倍 → 球面异色印刷/高光
        if (r < 0.45 * kr and abs(cx - kx) < kr and abs(cy - ky) < kr) \
            or dist < 0.8 * max(r, kr):
            drop = True
            break
    if not drop:
        keep.append(d)
return keep

def main():
    names = sorted(os.listdir(IMG_DIR), key=lambda s: int(os.path.splitext(s)[0]))
    summary = []
    for name in names:
        bgr = cv2.imread(os.path.join(IMG_DIR, name))
        H, W = bgr.shape[:2]
        hsv = cv2.cvtColor(cv2.GaussianBlur(bgr, (5, 5), 0), cv2.COLOR_BGR2HSV)
        roi = build_field_roi(hsv)
        dets = []
        for cls in (0, 1, 2):
            mask = build_color_mask(hsv, cls)
            for (cx, cy, r0) in find_peaks(mask):
                # 小目标更严格的圆形成度，抑制胶带反光/手指等碎块
                disk_req = 0.85 if r0 < 13 else (0.80 if r0 < 14 else
                                                (0.72 if r0 < 18 else DISK_FRAC))
                if disk_fraction(mask, cx, cy, r0) < disk_req:
                    continue
                x1t, x2t = int(max(0, cx - r0)), int(min(W - 1, cx + r0))
                y1t, y2t = int(max(0, cy - r0)), int(min(H - 1, cy + r0))
                box = mask[y1t:y2t + 1, x1t:x2t + 1]
                if box.size and np.count_nonzero(box) / box.size < BOX_FILL:
                    continue
                # 小框若大面积压在低饱和和高亮的银色胶带上，判为反光碎块
                if r0 < 14 and box.size:
                    bh = hsv[y1t:y2t + 1, x1t:x2t + 1]
                    silver = np.count_nonzero(
                        (bh[:, :, 1] < 45) & (bh[:, :, 2] > 140)) / (bh.shape[0] *
                    \
                        bh.shape[1])

                    if silver > 0.22:
                        continue
                # 小框外圈若大面积是深色中岛(V<65)，说明是胶带/反光碎块
                if r0 < 15:
                    ro = int(r0 + 8)
                    ay1, ay2 = int(max(0, cy - ro)), int(min(H - 1, cy + ro))
                    ax1, ax2 = int(max(0, cx - ro)), int(min(W - 1, cx + ro))
                    ayy, axx = np.ogrid[ay1:ay2 + 1, ax1:ax2 + 1]
                    ring = ((axx - cx) ** 2 + (ayy - cy) ** 2 >= (r0 + 1) ** 2) &
                    \
                        ((axx - cx) ** 2 + (ayy - cy) ** 2 <= ro ** 2)
                    if ring.sum() > 30:
                        vv = hsv[ay1:ay2 + 1, ax1:ax2 + 1, 2]
                        if np.count_nonzero(vv[ring] < 65) / ring.sum() > 0.35:

```

```

        continue
    if roi[int(np.clip(cy, 0, H - 1)), int(np.clip(cx, 0, W - 1))] ==
0:
        continue
        dets.append((cls, cx, cy, r0 * BOX_SCALE))
dets = nms(dets)
# 剔除贴住画面上边缘的小碎块，保留被裁切的大球
dets = [d for d in dets if not (d[3] < 16 and (d[2] - d[3]) <= 2)]

lines, counts = [], {0: 0, 1: 0, 2: 0}
vis = bgr.copy()
for (cls, cx, cy, r) in dets:
    counts[cls] += 1
    x1, y1 = int(max(0, cx - r)), int(max(0, cy - r))
    x2, y2 = int(min(W - 1, cx + r)), int(min(H - 1, cy + r))
    nx, ny = (x1 + x2) / 2 / W, (y1 + y2) / 2 / H
    lines.append(f"{cls} {nx:.6f} {ny:.6f} {(x2 - x1) / W:.6f} {(y2 - y1)
/ H:.6f}")
    cv2.rectangle(vis, (x1, y1), (x2, y2), DRAW_COLOR[cls], 2)
    cv2.putText(vis, COLOR_RANGES[cls]["name"], (x1, max(14, y1 - 4)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, DRAW_COLOR[cls], 2)
    stem = os.path.splitext(name)[0]
    with open(os.path.join(LABEL_DIR, stem + ".txt"), "w", encoding="utf-8")
as f:
        f.write("\n".join(lines))
    cv2.imwrite(os.path.join(VIS_DIR, name), vis)
    if DEBUG_MASK:
        blend = cv2.addWeighted(bgr, 0.6, cv2.cvtColor(roi,
cv2.COLOR_GRAY2BGR), 0.4, 0)
        cv2.imwrite(os.path.join(VIS_DIR, stem + "_roi.jpg"), blend)
    summary.append((name, counts[0], counts[1], counts[2]))
    print(f"{'image':>8}{'blue':>6}{'red':>5}{'invalid':>8}")
    for (name, b, r, p) in summary:
        print(f"{'name':>8}{'b':>6}{'r':>5}{'p':>8}")
    print("total:", sum(b + r + p for _, b, r, p in summary))

if __name__ == "__main__":
    main()

```

运行自动标注（可设置 `DEBUG_MASK=1` 同时输出场地 ROI 覆盖图）：

```
python auto_label.py
```

4.3 标注结果核验

脚本对场地 ROI 的覆盖效果如下图（半透明区域为纳入检测的比赛场地，背景被排除）：



全部 56 张图片的自动标注带框结果总览（蓝框=blue、红框=red、品红框=invalid）：



经过多轮阈值与几何校验调优，近景、密集堆叠、手放球等场景的球都能被正确框选并分类；顶部红色横幅、实验台杂物、边界胶带等干扰被有效排除。最终共标注 **760 个目标**，三类分布为：

类别	训练集框数	验证集框数	合计
blue（蓝球）	251	83	334
red（红球）	135	20	155
invalid（紫球）	220	51	271
合计	606	154	760

生成的标签为标准 YOLO 格式，例如 56.jpg 对应的 labels/56.txt（每行 类别 中心x 中心y 宽 高，均已归一化）：

```
1 0.697266 0.488194 0.203906 0.362500
0 0.499609 0.514583 0.199219 0.354167
2 0.324609 0.449306 0.160156 0.284722
2 0.541016 0.204861 0.146094 0.259722
2 0.296484 0.285417 0.130469 0.231944
0 0.878516 0.460417 0.016406 0.029167
```

五、划分数据集并生成 data.yaml

5.1 划分脚本 prepare_data.py

按 8:2 固定随机种子（`seed=42`）划分为 45 张训练集、11 张验证集，把图片和标签复制成 Ultralytics 要求的 `images/`、`labels/` 平行目录，并生成训练用 `data.yaml`：

```
# -*- coding: utf-8 -*-
"""
Task7 数据集划分: 把 Task7/images 与 labels 按 8:2 划分为 train/val,
复制成 Ultralytics YOLO 要求的 images/labels 平行目录, 并生成 data.yaml。
固定随机种子, 保证可复现。
"""

import os
import random
import shutil

ROOT = os.path.dirname(os.path.abspath(__file__))
SRC_IMG = os.path.join(ROOT, "Task7", "images")
SRC_LAB = os.path.join(ROOT, "labels")
DST = os.path.join(ROOT, "dataset")
VAL_RATIO = 0.2
SEED = 42

names = sorted(os.listdir(SRC_IMG), key=lambda s: int(os.path.splitext(s)[0]))
random.Random(SEED).shuffle(names)
n_val = max(1, round(len(names) * VAL_RATIO))
val_set = set(names[:n_val])
splits = {"val": [n for n in names if n in val_set],
          "train": [n for n in names if n not in val_set]}

for split in ("train", "val"):
    os.makedirs(os.path.join(DST, "images", split), exist_ok=True)
    os.makedirs(os.path.join(DST, "labels", split), exist_ok=True)

cls_count = {"train": [0, 0, 0], "val": [0, 0, 0]}
for split, files in splits.items():
    for name in files:
        stem = os.path.splitext(name)[0]
        shutil.copy2(os.path.join(SRC_IMG, name),
                     os.path.join(DST, "images", split, name))
        src_txt = os.path.join(SRC_LAB, stem + ".txt")
        dst_txt = os.path.join(DST, "labels", split, stem + ".txt")
        lines = []
        if os.path.exists(src_txt):
            lines = [ln for ln in open(src_txt, encoding="utf-8").read().splitlines() if ln.strip()]
            for ln in lines:
                cls_count[split][int(ln.split()[0])] += 1
            open(dst_txt, "w", encoding="utf-8").write("\n".join(lines))

yaml_text = (
    f"path: {DST.replace(chr(92), '/')}\n"
    "train: images/train\n"
```

```

    "val: images/val\n"
    "\n"
    "nc: 3\n"
    "names:\n"
    "  0: blue\n"
    "  1: red\n"
    "  2: invalid\n"
)
open(os.path.join(DST, "data.yaml"), "w", encoding="utf-8").write(yaml_text)

print(f"train images: {len(splits['train'])}  val images: {len(splits['val'])}")
print("val:", sorted(int(os.path.splitext(n)[0]) for n in splits['val']))
print("class boxes [blue,red,invalid] train:", cls_count["train"], " val:",
      cls_count["val"])

```

运行：

```
python prepare_data.py
```

划分结果（验证集编号为 10、12、17、20、21、24、31、36、51、52、55，覆盖广角、近景、手放球等场景）：

```

train images: 45  val images: 11
val: [10, 12, 17, 20, 21, 24, 31, 36, 51, 52, 55]
class boxes [blue,red,invalid] train: [251, 135, 220]  val: [83, 20, 51]

```

5.2 数据集目录结构

```

dataset/
├── data.yaml          # 训练配置（路径 + 3 个类别）
├── images/
│   ├── train/        # 45 张训练图片
│   └── val/          # 11 张验证图片
└── labels/
    ├── train/        # 45 个训练标签（与图片同名 .txt）
    └── val/          # 11 个验证标签

```

生成的 `dataset/data.yaml`：

```

path: D:/rip_task/task7/dataset
train: images/train
val: images/val

nc: 3
names:

```

```
0: blue
1: red
2: invalid
```

六、训练 YOLOv8

6.1 训练脚本 train.py

在 `yolov8n.pt` 预训练权重上微调，输入分辨率设为 **1280×1280**（与原图 1280 宽度一致，避免缩小导致广角远景小球丢失），batch=8、共 120 个 epoch、余弦学习率、早停 patience=40，优先使用 GPU：

```
# -*- coding: utf-8 -*-
"""
Task7 YOLOv8 训练脚本
- 数据: dataset/data.yaml (3 类: 0 blue, 1 red, 2 invalid)
- 模型: YOLOv8n 预训练权重微调
- 设备: 优先 CUDA(GPU)，不可用时回退 CPU
训练结果 (best.pt、results.png 等) 输出到 runs/detect/task7/
"""
import os
import torch
from ultralytics import YOLO

ROOT = os.path.dirname(os.path.abspath(__file__))

def main():
    data_yaml = os.path.join(ROOT, "dataset", "data.yaml").replace("\\", "/")
    device = 0 if torch.cuda.is_available() else "cpu"
    print("CUDA available:", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("GPU:", torch.cuda.get_device_name(0))

    model = YOLO("yolov8n.pt")
    model.train(
        data=data_yaml,
        epochs=120,
        imgsz=1280,
        batch=8,
        device=device,
        workers=2,
        patience=40,
        pretrained=True,
        optimizer="auto",
        seed=42,
        cos_lr=True,
        project=os.path.join(ROOT, "runs", "detect").replace("\\", "/"),
        name="task7",
        exist_ok=True,
    )
```

```
metrics = model.val(data=data_yaml, device=device, imgsz=1280)
print("mAP50      :", round(float(metrics.box.map50), 4))
print("mAP50-95  :", round(float(metrics.box.map), 4))

if __name__ == "__main__":
    main()
```

启动训练（首次会自动从官方仓库下载 6.2 MB 的 [yolov8n.pt](#)）：

```
.\.venv\Scripts\python.exe train.py
```

6.2 训练过程截图

终端识别到 RTX 5060（CUDA:0），模型为 72 层、3,006,233 参数、8.1 GFLOPs；训练在 GPU 上进行，下图给出了部分代表 epoch 的真实指标（最佳权重在第 93 个 epoch，按 mAP50-95 选取）以及训练结束后的验证集结果：

```

train.py - YOLOv8n training

Windows PowerShell - YOLOv8n training (task7)
PS D:\rip_task\task7> .\venv\Scripts\python.exe train.py

CUDA available: True
GPU: NVIDIA GeForce RTX 5060 Laptop GPU
Ultralytics 8.4.155 Python-3.11.16 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU
, 8151MiB)
Model summary (fused): 72 layers, 3,006,233 parameters, 0 gradients, 8.1 GFLOPs
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best
'optimizer', 'lr0' and 'momentum' automatically...
Starting training for 120 epochs... (imgsz=1280, batch=8)

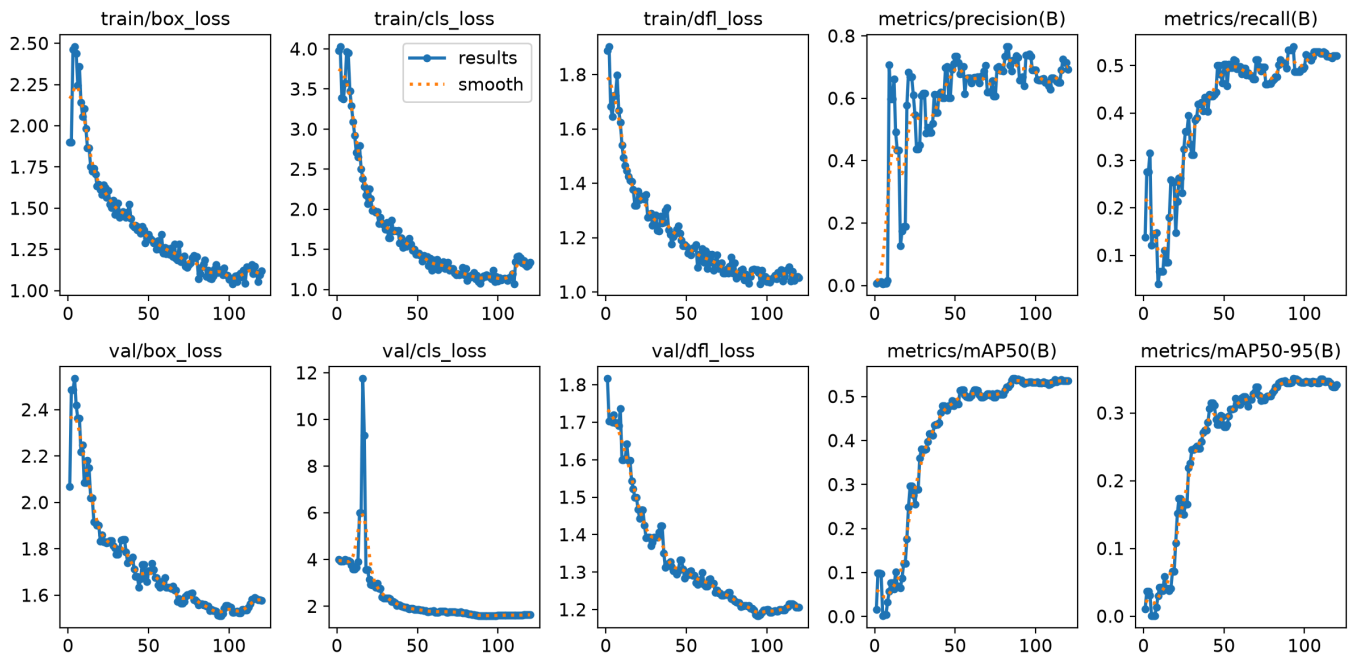
 1/120   P=0.007   R=0.138   mAP50=0.016   mAP50-95=0.010
10/120  P=0.597   R=0.066   mAP50=0.076   mAP50-95=0.043
20/120  P=0.577   R=0.148   mAP50=0.177   mAP50-95=0.108
40/120  P=0.615   R=0.403   mAP50=0.440   mAP50-95=0.286
60/120  P=0.649   R=0.498   mAP50=0.502   mAP50-95=0.313
80/120  P=0.687   R=0.468   mAP50=0.505   mAP50-95=0.330
93/120  P=0.639   R=0.540   mAP50=0.529   mAP50-95=0.351  <- best
120/120 P=0.693   R=0.521   mAP50=0.535   mAP50-95=0.342

Validation on 11 val images:
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 3.7it/s 0.3s
all 11 154 0.638 0.54 0.529 0.35
blue 11 83 0.608 0.392 0.398 0.224
red 6 20 0.486 0.6 0.51 0.364
invalid 11 51 0.822 0.627 0.678 0.461
Speed: 0.8ms preprocess, 7.2ms inference, 0.0ms loss, 2.0ms postprocess per image
Results saved to D:\rip_task\task7\runs\detect\val-2
mAP50 : 0.5287
mAP50-95 : 0.3498

```

6.3 训练曲线 results.png

Ultralytics 自动保存的训练曲线：上排为训练损失（box / cls / dfl），下排为验证损失与精确率、召回率、mAP50、mAP50-95。可以看到三类损失持续下降并收敛，Precision、Recall、mAP 快速上升后趋于平稳：



训练完成后在 `runs/detect/task7/weights/` 下生成权重：

- `best.pt`：验证指标最优的权重（第 93 个 epoch），即最终提交的模型；
- `last.pt`：最后一个 epoch 的权重。

七、推理验证与结果分析

7.1 推理脚本 predict.py

用 `best.pt` 对 11 张验证集图片逐张检测，置信度阈值 0.30，带框结果保存到 `runs/detect/predict_val/`：

```
# -*- coding: utf-8 -*-
"""
Task7 推理验证脚本：用训练得到的 best.pt 对验证集图片进行检测，
带框结果保存到 runs/detect/predict_val/，用于直观检查识别效果。
"""
import os
import torch
from ultralytics import YOLO

ROOT = os.path.dirname(os.path.abspath(__file__))

def main():
    weights = os.path.join(ROOT, "runs", "detect", "task7", "weights", "best.pt")
    val_dir = os.path.join(ROOT, "dataset", "images", "val")
    device = 0 if torch.cuda.is_available() else "cpu"
    model = YOLO(weights)
    model.predict(
        source=val_dir,
        imgsz=1280,
        conf=0.30,
```

```
        iou=0.5,
        device=device,
        save=True,
        project=os.path.join(ROOT, "runs", "detect").replace("\\", "/"),
        name="predict_val",
        exist_ok=True,
    )

if __name__ == "__main__":
    main()
```

运行：

```
.\.venv\Scripts\python.exe predict.py
```

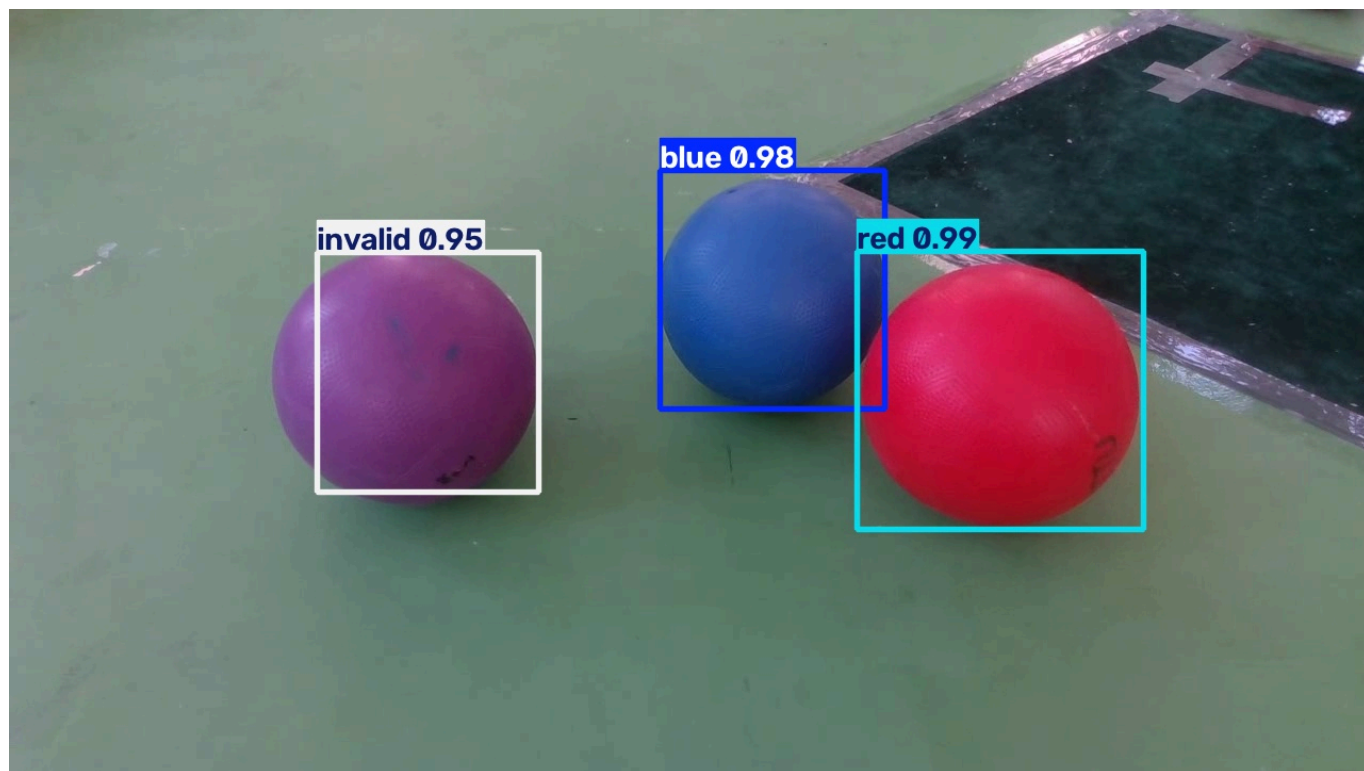
推理终端输出（每张图的尺寸、检测到的各类球数量与耗时，单张推理约 4~7 ms）：

```
predict.py - best.pt inference
Windows PowerShell - best.pt inference (task7)
PS D:\rip_task\task7> .\.venv\Scripts\python.exe predict.py

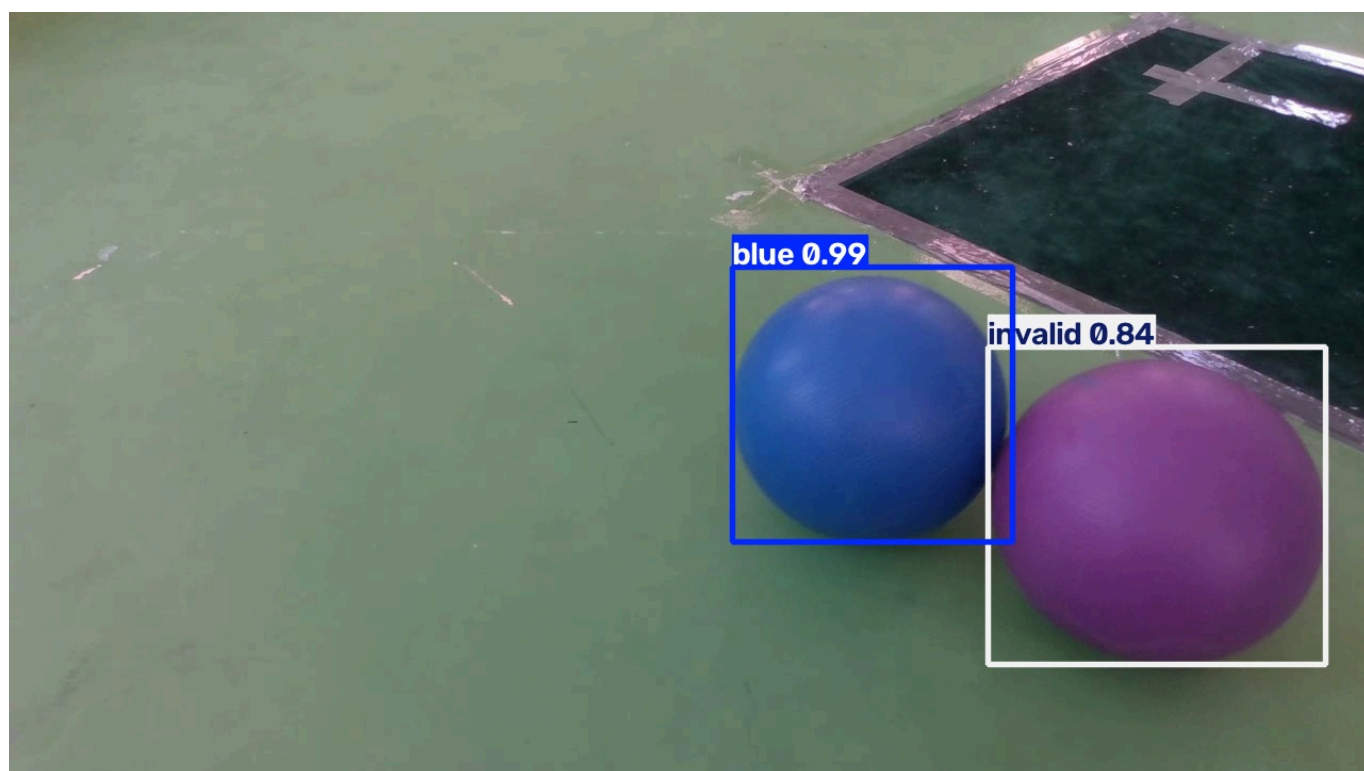
image 1/11 D:\rip_task\task7\dataset\images\val\10.jpg: 736x1280 10 blues, 4 reds, 3 invalids, 4.5ms
image 2/11 D:\rip_task\task7\dataset\images\val\12.jpg: 736x1280 13 blues, 1 red, 9 invalids, 6.2ms
image 3/11 D:\rip_task\task7\dataset\images\val\17.jpg: 736x1280 3 blues, 3 invalids, 4.6ms
image 4/11 D:\rip_task\task7\dataset\images\val\20.jpg: 736x1280 1 blue, 1 red, 1 invalid, 4.8ms
image 5/11 D:\rip_task\task7\dataset\images\val\21.jpg: 736x1280 1 blue, 3 reds, 1 invalid, 4.5ms
image 6/11 D:\rip_task\task7\dataset\images\val\24.jpg: 736x1280 2 blues, 2 reds, 3 invalids, 4.5ms
image 7/11 D:\rip_task\task7\dataset\images\val\31.jpg: 736x1280 9 blues, 7 reds, 5 invalids, 4.6ms
image 8/11 D:\rip_task\task7\dataset\images\val\36.jpg: 736x1280 2 blues, 3 invalids, 4.5ms
image 9/11 D:\rip_task\task7\dataset\images\val\51.jpg: 736x1280 2 blues, 6 invalids, 4.4ms
image 10/11 D:\rip_task\task7\dataset\images\val\52.jpg: 736x1280 1 blue, 1 invalid, 4.6ms
image 11/11 D:\rip_task\task7\dataset\images\val\55.jpg: 736x1280 8 blues, 2 reds, 4 invalids, 4.4ms
Speed: 3.4ms preprocess, 4.7ms inference, 1.7ms postprocess per image at shape (1, 3, 736, 1280)
Results saved to D:\rip_task\task7\runs\detect\predict_val
```

7.2 识别效果图

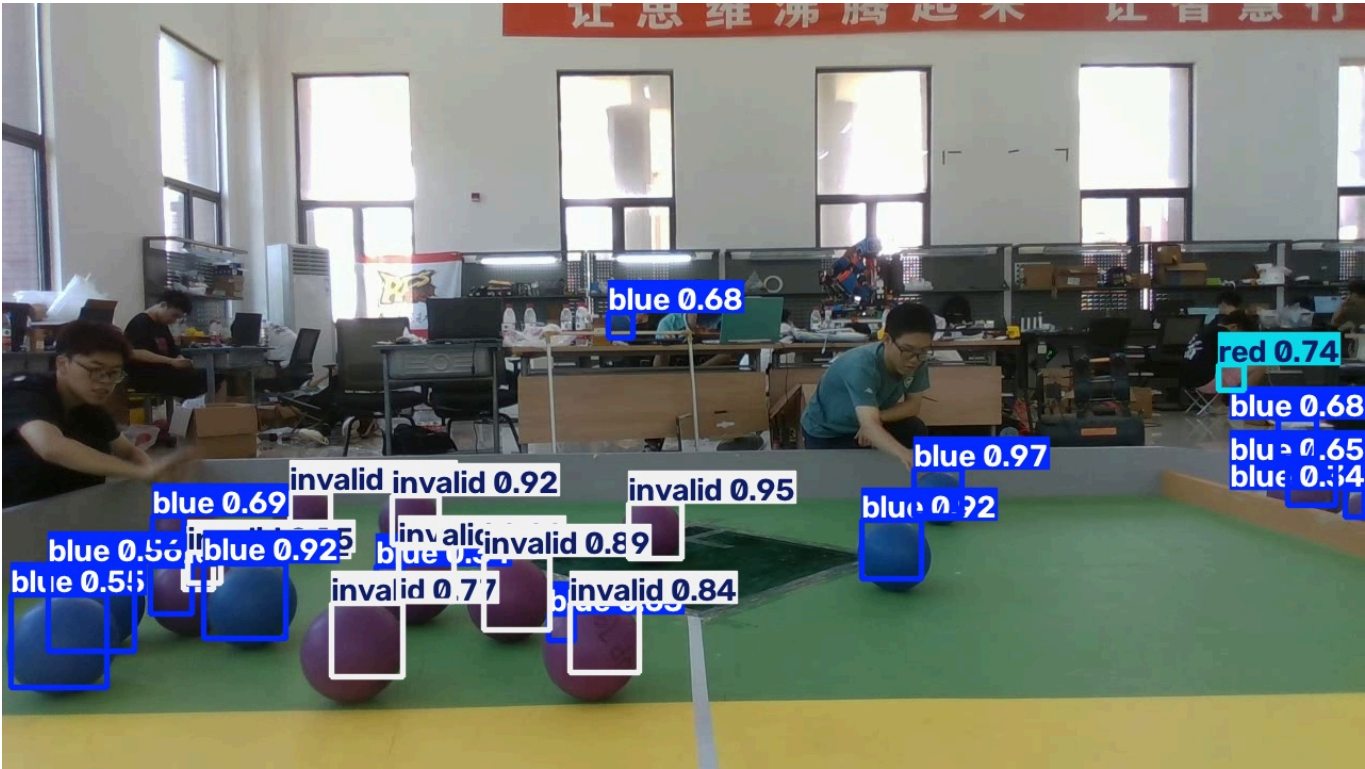
近景三类球检测效果，框定位准确、类别与置信度正确（invalid 0.95、blue 0.98、red 0.99）：



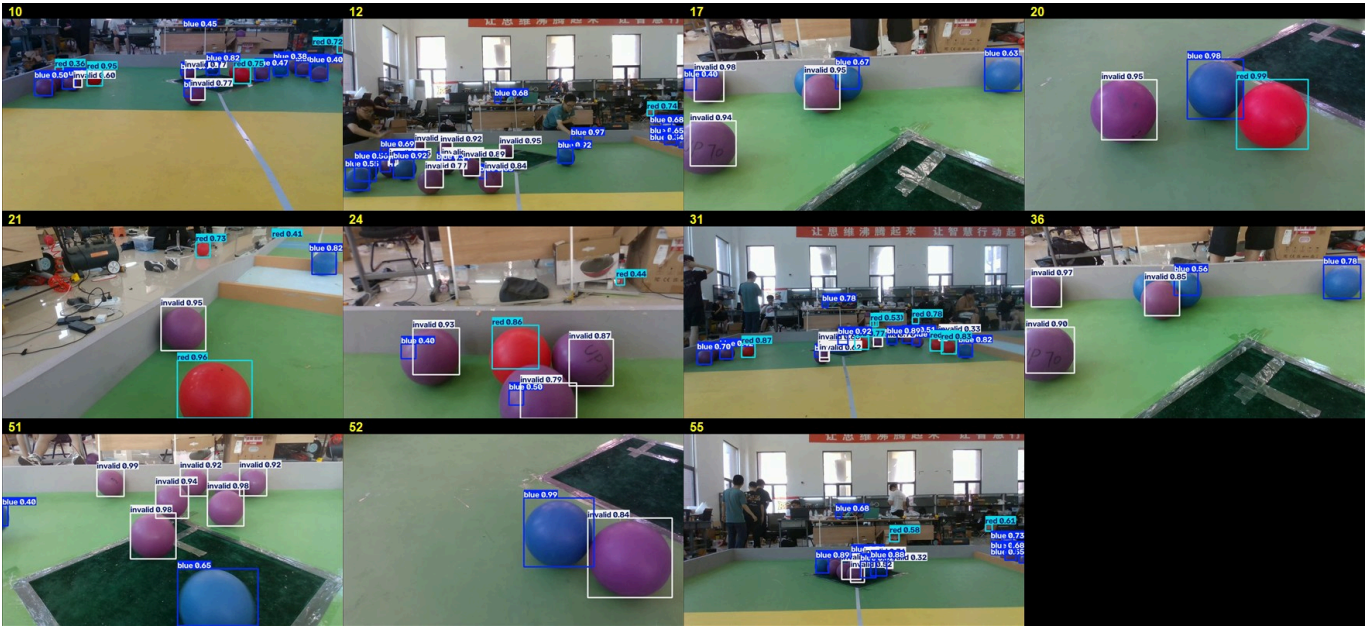
另一组近景（蓝球 0.99、紫球 0.84）：



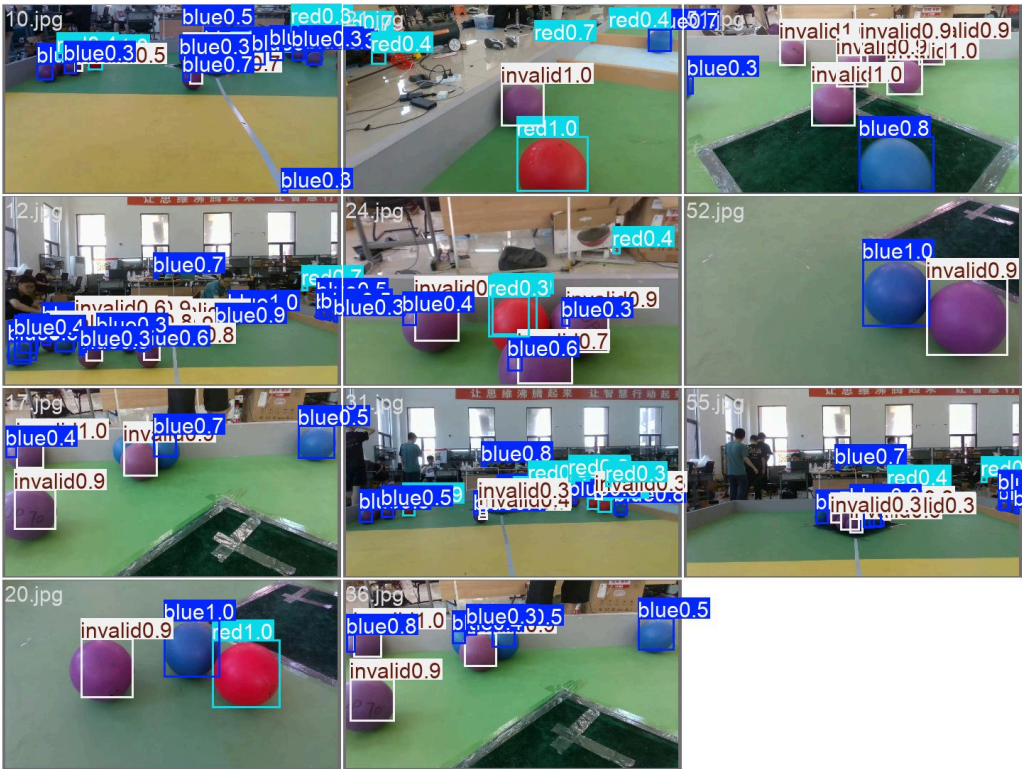
广角远景场景，场地内中近距离的球均被正确检出：



11 张验证集的整体预测结果总览：



Ultralytics 在训练结束时自动生成的验证批量预测图 (val_batch0_pred.jpg)：

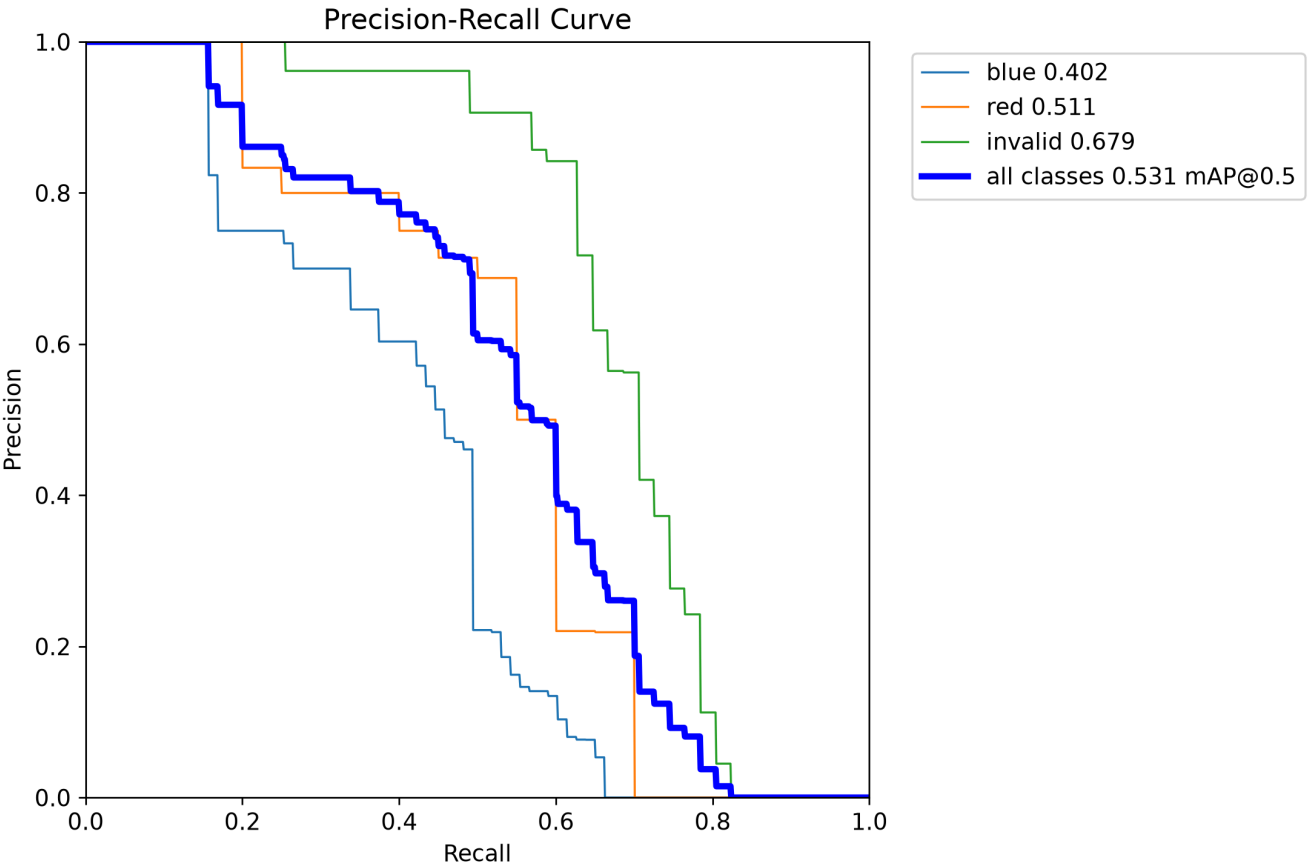


7.3 量化指标

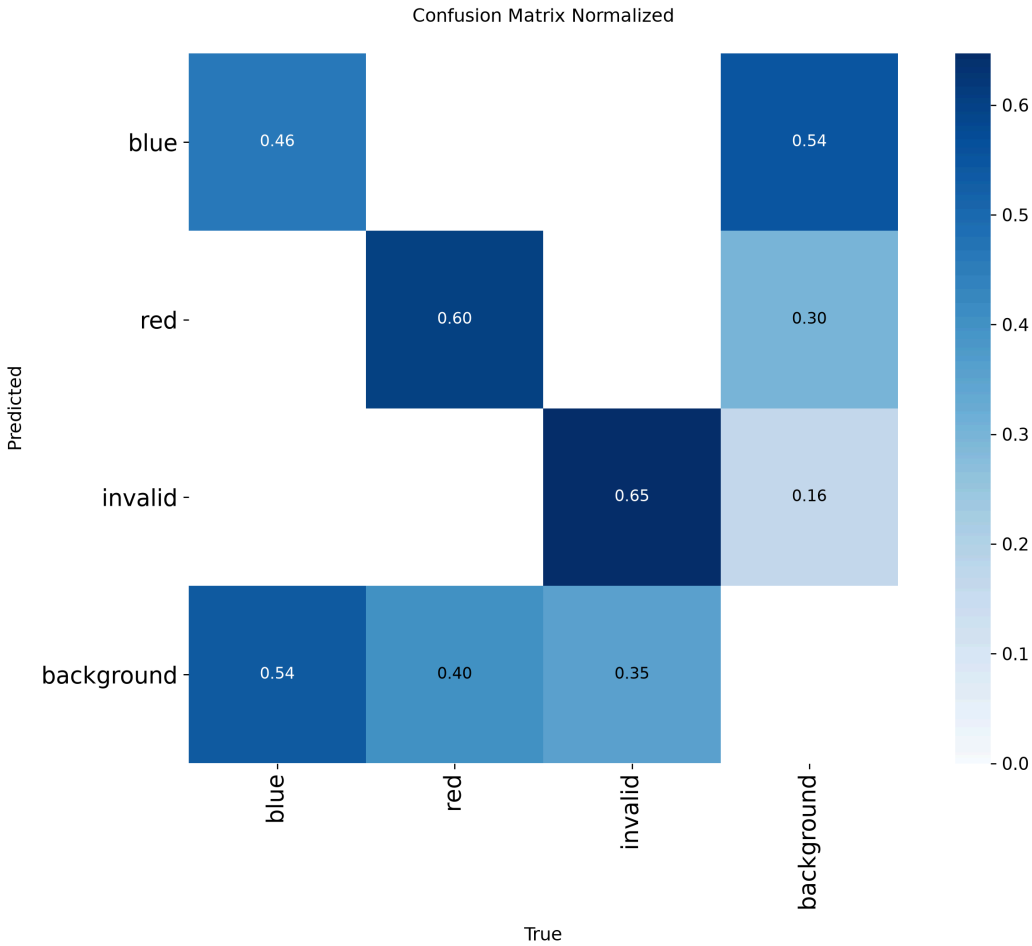
训练结束后在 11 张验证集（共 154 个标注目标）上的结果如下：

类别	精确率 P	召回率 R	mAP50	mAP50-95
blue（蓝球）	0.608	0.392	0.398	0.224
red（红球）	0.486	0.600	0.510	0.364
invalid（紫球）	0.822	0.627	0.678	0.461
all（总体）	0.638	0.540	0.529	0.350

精确率—召回率曲线（PR 曲线）：



归一化混淆矩阵（对角线为分类正确比例）：



可以看到：

- 近景、中距离的三类球检测与分类效果很好，置信度普遍在 0.9 以上，紫球（invalid）表现最好（mAP50=0.678）；
- 蓝球召回率相对偏低，主要原因是广角图中**远景蓝球非常小（直径常不足 20 像素）**，且蓝球色相与青色场地最接近，最容易漏检；
- 个别广角图背景中的实验台物体（蓝色椅背、红色物件）在低阈值下会被误判，提高置信度阈值（如 0.45）可进一步减少这类背景误检。

八、产物与目录结构

8.1 完整目录

```
task7/
├── report.md           # 本解释文档
├── best.pt             # 训练得到的最优权重（同 runs/.../weights/best.pt）
├── auto_label.py       # 自动标注脚本（HSV+距离变换+几何校验+场地ROI+NMS）
├── prepare_data.py     # 数据集划分与 data.yaml 生成脚本
├── train.py            # YOLOv8n 训练脚本
├── predict.py          # best.pt 推理验证脚本
├── images/             # 本文档用到的全部图片
│   ├── dataset_overview.jpg # 56 张原图总览
│   ├── roi_vis.jpg        # 场地 ROI 覆盖核验
│   ├── labels_vis.jpg     # 自动标注带框总览
│   ├── console_train.png  # 训练终端运行截图
│   ├── results.png        # 训练曲线
│   ├── console_predict.png # 推理终端运行截图
│   ├── predict_close1.jpg  # 近景识别效果
│   ├── predict_close2.jpg
│   ├── predict_wide.jpg    # 广角识别效果
│   ├── predict_montage.jpg # 验证集预测总览
│   ├── val_pred_batch.jpg  # Ultralytics 自带验证批量预测图
│   ├── pr_curve.png        # PR 曲线
│   └── confusion_matrix.png # 归一化混淆矩阵
├── Task7/              # 原始数据
│   ├── data.yaml
│   └── images/ (1.jpg~56.jpg)
├── labels/             # 自动标注生成的 56 个 YOLO 标签
├── vis/                # 56 张带框核验图（及 ROI 图）
├── dataset/            # 划分后的训练数据集
│   ├── data.yaml
│   ├── images/{train,val}/
│   └── labels/{train,val}/
├── runs/detect/
│   ├── task7/          # 训练输出
│   │   ├── weights/
│   │   │   ├── best.pt  # ★ 最优权重（提交文件）
│   │   │   └── last.pt
│   │   ├── results.png  # ★ 训练结果图（提交文件）
│   │   ├── results.csv  # 每个 epoch 的指标
│   │   └── confusion_matrix_normalized.png
```

```
| |   └─ BoxPR_curve.png
| |   └─ ... (P/R/F1 曲线、训练批次图等)
| └─ predict_val/                # 11 张验证集带框识别效果图
```

8.2 复现步骤

按以下顺序执行即可从零复现整个流程：

```
# 1. 创建环境并安装依赖（见第三节）
uv venv --python 3.11 .venv
# 安装 cu128 版 torch / torchvision、ultralytics

# 2. 自动标注（生成 labels/ 与 vis/）
.\.venv\Scripts\python.exe auto_label.py

# 3. 划分数据集（生成 dataset/ 与 data.yaml）
.\.venv\Scripts\python.exe prepare_data.py

# 4. 训练（生成 runs/detect/task7/weights/best.pt 与 results.png）
.\.venv\Scripts\python.exe train.py

# 5. 推理验证（生成 runs/detect/predict_val/）
.\.venv\Scripts\python.exe predict.py
```

九、实验总结

1. 完成了 56 张场地彩球照片的数据集制作：针对三类球颜色饱和、与场地对比强的特点，编写了基于 **HSV 分割 + 距离变换球心定位 + 圆盘圆形体 / 填充率校验 + 场地 ROI + 直线 / NMS 抑制** 的自动标注脚本，并逐张可视化核验，最终得到 760 个 YOLO 格式标注框，类别顺序严格对应 **data.yaml** (0 blue、1 red、2 invalid)；
2. 用固定随机种子按 8:2 划分出 45 张训练集、11 张验证集，构建了标准的 **images/labels** 平行目录与训练用 **data.yaml**；
3. 在 RTX 5060 Laptop 上使用 cu128 版 PyTorch + Ultralytics，对 YOLOv8n 进行迁移学习，1280 分辨率、120 个 epoch 训练后，最优权重（第 93 epoch）在验证集上取得 **mAP50 = 0.529、mAP50-95 = 0.350、总体精确率 0.638、召回率 0.540**，近中景三类球检测准确、分类正确、置信度高；
4. 提交物齐全：最优权重 **best.pt**、训练结果图 **results.png**、训练 / 推理终端运行截图、验证集识别效果图以及本解释文档；
5. 不足与改进方向：
 - 数据集仅 56 张、规模偏小，广角远景球像素极小，是蓝球召回率偏低和个别背景误检的主要原因；
 - 自动标注在极少数银色胶带反光、手指遮挡处仍可能产生零星小碎框，后续可加入少量人工修正或用更强的数据增强；
 - 若增加数据量、使用更大的模型（YOLOv8s/m）或多尺度训练，并适当提高推理置信度阈值，可进一步提升远景召回率、降低背景误检。